

1 Summary

2 Scientific workflows require coordination across manuscript preparation,
3 computational analysis, figure generation, data management, and literature
4 curation, yet these activities remain distributed across disconnected tools,
5 making reproducibility difficult to sustain. SciTeX is an AI-native, mod-
6 ular research platform that unifies these tasks through four core compo-
7 nents—Writer, Scholar, Viz, and Code—with optional extensions for statis-
8 tical analysis. Each module provides standalone functionality while partici-
9 pating in a shared project structure built on lightweight symlinks and global
10 identifiers, enabling provenance-preserving links between code, data, figures,
11 and manuscript text. Containerized builds ensure byte-consistent manuscript
12 output across systems, and standardized metadata captures computational
13 and visual parameters for downstream verification or regeneration. By em-
14 bedding reproducibility infrastructure directly into routine authoring, visual-
15 ization, and analysis workflows, SciTeX lowers the activation energy required
16 for transparent scientific practice while maintaining compatibility with ex-
17 isting toolchains. This manuscript, prepared and compiled entirely within
18 SciTeX, demonstrates its end-to-end integration.

19 Highlights

- 20 • SciTeX unifies manuscript writing, computation, figure generation, and
21 literature management in a single reproducible research environment.
- 22 • Modular design allows each component to function independently while
23 enabling seamless integration through lightweight symlinks and global
24 identifiers.
- 25 • Containerized LaTeX builds and standardized metadata formats ensure
26 consistent manuscripts and reconstructible figures across systems.
- 27 • Provenance-preserving workflows link code, data, figures, and text, re-
28 ducing cognitive load and supporting transparent scientific practice.

- 29 • AI-native architecture exposes structured interfaces for automated anal-
30 ysis, figure refinement, and manuscript assistance.

SciTeX: A unified research OS for reproducible scientific workflows

Yusuke Watanabe^{a,*}

^a*SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

~ 8 figures, 3 tables, 146 words for abstract, and 2542 words for main text

1. Introduction

Scientific workflows increasingly span heterogeneous environments, including personal laptops, cloud servers, containerized pipelines, and HPC systems. Yet the core activities of research—manuscript writing, computational analysis, figure generation, and literature management—remain distributed across disconnected tools. This fragmentation increases cognitive load, introduces opportunities for error, and makes reproducibility difficult to sustain [1, 2]. Researchers must manually coordinate figures across formats, maintain consistent bibliographies, track versions of manuscripts and data, and ensure that computational analyses remain synchronized with the text that describes them [3, 4]. These burdens often overshadow the substantive scientific work.

Traditional LaTeX-based manuscript preparation further amplifies these challenges. Local installations vary across machines and over time, producing inconsistent builds and complicating collaboration [5]. Figure generation typically occurs in isolation from manuscript writing, with libraries and scripts lacking standardized metadata or provenance tracking. Literature manage-

*Corresponding author. Email: ywatanabe@scitex.ai

56 ment tools such as Zotero and Mendeley help organize references but oper-
57 ate independently of manuscript preparation and computational workflows,
58 creating citation drift when documents evolve. Computational analyses ex-
59 ecuted in Jupyter notebooks or ad hoc scripts provide convenience but lack
60 the structured reproducibility required for long-term verification [6]. As a
61 result, researchers must orchestrate a complex toolchain whose components
62 have no shared model of provenance, reproducibility, or workflow state.

63 Existing solutions address individual components of this ecosystem but
64 stop short of providing a unified workflow. Overleaf ensures consistent La-
65 TeX compilation but requires separate figure generation and data manage-
66 ment workflows. Visualization tools such as SigmaPlot and Origin offer pol-
67 ished interfaces but remain disconnected from manuscript and code envi-
68 ronments. Manubot demonstrates the value of Git-based open writing [7],
69 while Binder and related technologies provide reproducible computational
70 environments [8]. Still, no existing platform integrates writing, computation,
71 visualization, and literature management in a way that preserves provenance
72 and reduces cognitive overhead [9, 10].

73 SciTeX addresses this gap by providing a unified, modular, AI-native
74 platform for scientific research. Its Writer module offers structured La-
75 TeX authoring with fully containerized compilation; Scholar manages cita-
76 tion retrieval, metadata extraction, and PDF organization; Viz produces
77 publication-quality figures with embedded provenance and reconstructible
78 metadata; and Code supports reproducible computation with GPU and con-
79 tainer integration. A shared project structure built on lightweight sym-
80 links and global identifiers links these modules, enabling consistent cross-
81 referencing between data, figures, code, and manuscript text. Researchers
82 can adopt modules independently while benefiting from deeper integration
83 as their workflows evolve.

84 By embedding reproducibility principles directly into everyday research
85 tools [11], SciTeX lowers the activation energy required for rigorous scientific
86 practice. It ensures that manuscripts and computational outputs remain

87 synchronized, that figures can be regenerated from metadata, and that bibli-
88 ographic consistency is maintained automatically. This manuscript, authored
89 and compiled entirely within SciTeX, serves as a self-descriptive demonstra-
90 tion of the platform’s design goals and integrated capabilities.

91 **2. Methods**

92 *2.1. System Architecture*

93 SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and
94 a Python package (v2.4.1+) installable via `pip install scitex`. The ar-
95 chitecture employs lazy module loading via a custom `_LazyModule` wrapper,
96 enabling fast imports while providing access to over 50 specialized mod-
97 ules. This modular design follows best practices for scientific Python appli-
98 cations [12, 1]:

```
99 import scitex as stx
100 stx.plt          # Publication plotting
101 stx.io           # Universal file I/O (30+ formats)
102 stx.scholar      # Literature management
103 stx.session      # Experiment lifecycle
104 stx.vis          # Visual figure specifications
105 stx.writer       # Manuscript preparation
```

106 Each module operates independently while sharing a unified project struc-
107 ture through lightweight symlinks. Global identifiers (SHA256 hashes) ensure
108 traceability between figures, code, bibliography entries, and manuscript con-
109 tent, enabling reverse lookup capabilities (e.g., “which script generated this
110 figure?”). This approach addresses concerns raised in discussions of figure
111 provenance and data reproducibility [3, 13].

112 *2.2. Session Decorator for Reproducible Experiments*

113 The `@stx.session` decorator provides automatic experiment lifecycle
114 management, implementing principles from reproducible research frameworks [2,
115 14]:

```

116 import scitex as stx
117
118 @stx.session
119 def analyze(data_path: str, threshold: float = 0.5):
120     '''Analyze data file.'''
121     data = stx.io.load(data_path)
122     result = process(data, threshold)
123     stx.io.save(result, "output.csv")
124     return 0
125
126 if __name__ == '__main__':
127     analyze() # CLI: python script.py --data-path x --threshold 0.7

```

128 The decorator automatically: (1) generates CLI argument parsers from
129 function signatures with type hints, (2) initializes session directories with
130 timestamped run IDs, (3) injects global variables (`CONFIG`, `plt`, `COLORS`,
131 `rng_manager`, `logger`), (4) manages random state for reproducibility, and
132 (5) handles cleanup and optional notifications on completion. This design is
133 informed by the “good enough practices” paradigm [1] and project manage-
134 ment tools for team science [15].

135 *2.3. Publication-Quality Plotting (stx.plt)*

136 The `stx.plt` module wraps matplotlib with automatic configuration from
137 `SCITEX_STYLE.yaml`:

```

138 import scitex.plt as splt
139
140 fig, ax = splt.subplots(axes_width_mm=40, axes_height_mm=28)
141 ax.plot([1, 2, 3], [1, 4, 9])
142 ax.set_xyt("Time", "Value", "Analysis Results")
143 stx.io.save(fig, "figure.png") # Exports PNG + JSON + CSV

```

144 Key features include: (1) automatic style application on import (font
145 sizes, line widths, spine visibility), (2) `FigWrapper` and `AxisWrapper` classes
146 providing `set_xyt()` for simultaneous x-label, y-label, and title setting, (3)
147 automatic CSV export of underlying plot data alongside figures, (4) JSON
148 sidecar metadata encoding axis bounds, tick locations, panel geometry, and
149 color palettes, and (5) `stx.plt.load()` for figure reconstruction from saved
150 metadata. This triple-export approach (image, metadata, data) addresses
151 the reproducibility challenges identified in computational figure generation [3,
152 4].

153 Style values can be customized via YAML configuration, environment
154 variables (`SCITEX_PLT_FONTS_AXIS_LABEL_PT=8`), or direct function param-
155 eters.

156 2.4. Universal File I/O (*stx.io*)

157 The `stx.io` module provides format-agnostic file operations supporting
158 over 30 formats, following the principle of unified interfaces for data man-
159 agement [16]:

```
160 import scitex as stx
161
162 # Automatic format detection
163 data = stx.io.load("data.csv")      # DataFrame
164 config = stx.io.load("config.yaml") # Dict
165 model = stx.io.load("model.pkl")    # Python object
166 arrays = stx.io.load("data.h5")     # HDF5 exploration
167
168 # Unified save interface
169 stx.io.save(df, "output.csv")
170 stx.io.save(fig, "figure.png")      # PNG + JSON + CSV
171 stx.io.save(config, "config.yaml")
```

172 Supported formats include: CSV, JSON, YAML, Pickle, HDF5, Zarr,
173 NumPy arrays, PyTorch tensors, images (PNG, JPG, SVG, PDF), audio,

174 video, and MATLAB files. The module includes `H5Explorer` and `ZarrExplorer`
175 for interactive hierarchical data navigation, plus configurable caching for fre-
176 quently accessed files.

177 2.5. Literature Management (*stx.scholar*)

178 The Scholar module provides multi-source literature search and manage-
179 ment, integrating capabilities typically found in standalone reference man-
180 agers:

```
181 from scitex.scholar import Scholar
182
183 scholar = Scholar()
184 papers = scholar.search("deep learning", year_min=2022)
185 papers.save("bibliography.bib")
```

186 The module integrates with multiple metadata engines (CrossRef, Sema-
187 tic Scholar, OpenAlex) via the `ScholarEngine` interface. Features include:
188 automatic DOI detection and metadata extraction from PDFs, `ScholarPDFDownloader`
189 for paper acquisition, `ScholarLibrary` for local storage management, and
190 `Paper/Papers` classes with filtering and export capabilities. This design
191 complements tools like Manubot [7] by providing direct integration with
192 manuscript preparation.

193 2.6. Cloud Platform Architecture

194 The Django 5.1-based cloud platform provides web interfaces for all mod-
195 ules, following principles of continuous integration for scientific publishing [17]:

196 **Code App:** Browser-based Python execution with real-time output stream-
197 ing, file upload/download, and workspace management.

198 **Viz App:** Canvas-based figure editor using Fabric.js 5.5.2 for interactive
199 element manipulation, with JSON specification import/export for program-
200 matic control.

201 **Writer App:** Structured LaTeX manuscript preparation with container-
202 ized compilation (Docker/Apptainer), section-separated authoring, and au-
203 tomatic figure format conversion.

204 **Scholar App:** Literature cards with abstract preview, PDF access,
205 metadata editing, and direct citation insertion into Writer documents.

206 The platform employs TypeScript for frontend logic, PostgreSQL for data
207 persistence, and Docker for containerized compilation ensuring consistent
208 output across environments.

209 2.7. Containerized Compilation

210 Writer module compilation leverages containerization for reproducibility,
211 implementing guidelines for Docker-based scientific workflows [5, 18]:

212 **Multi-Engine Support:** Tectonic [19] for ultra-fast incremental builds
213 (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 sec-
214 onds), and traditional 3-pass compilation for maximum compatibility.

215 **Environment Isolation:** Docker and Apptainer/Singularity containers
216 encapsulate specific TeX Live versions and all required packages, eliminat-
217 ing host system dependencies and guaranteeing identical PDF output across
218 macOS, Linux, Windows, and HPC environments [20, 21].

219 **Version Tracking:** Git integration [22] with automatic latexdiff gener-
220 ation facilitates peer review processes [23].

221 2.8. AI-Native Workflow Integration

222 SciTeX provides native integration points for AI-assisted workflows, an-
223 ticipating the growing role of AI in scientific research [24, 25]:

224 **Structured APIs:** JSON-based figure specifications, typed function sig-
225 natures for CLI generation, and metadata formats amenable to automated
226 analysis enable LLM-assisted code generation and figure improvement.

227 **Workflow Hooks:** The session decorator and modular architecture pro-
228 vide hooks for AI-assisted revision, automated citation recommendation, and
229 semantic search across analyses and literature.

230 **Research Co-pilot Vision:** The architecture supports future devel-
231 opment of automated analysis pipelines, figure generation, and manuscript
232 drafting while maintaining researcher oversight and scientific integrity [26].

233 2.9. Implementation Details

234 The system supports deployment on local machines, cloud servers (AWS,
235 GCP, Azure), and self-hosted NAS environments. Key technical specifica-
236 tions:

- 237 • Python 3.12+ with GPU acceleration support (CUDA 11.x/12.x)
- 238 • Django 5.1 with PostgreSQL backend
- 239 • TypeScript frontend with webpack bundling
- 240 • Docker/Apptainer containerization for reproducible execution [5]
- 241 • SLURM integration under active development for HPC workflows

242 The platform follows FAIR principles [27] and aligns with the Turing Way
243 guidelines for reproducible research [11].

244 3. Results

245 This section demonstrates the capabilities of SciTeX through validation
246 of its core modules and their behavior in realistic research workflows. The
247 goal is not to present experimental findings but to illustrate how the platform
248 operationalizes reproducibility, provenance tracking, and integrated author-
249 ing.

250 3.1. Reproducible Execution with the Session Decorator

251 The `@stx.session` decorator reliably transforms Python functions into
252 traceable command-line experiments. Across multiple test scenarios, SciTeX
253 generated timestamped session directories that captured parameters, logs,

254 outputs, and metadata. Automatically injected globals—including a repro-
255 ducible random number manager, configuration state, logging interface, and
256 plotting environment—enabled functions to execute without boilerplate. Re-
257 peated runs produced consistent outputs when supplied with identical param-
258 eters, confirming correct synchronization of random state and environment
259 settings. Errors were handled gracefully while preserving intermediate out-
260 puts, supporting transparent debugging.

261 *3.2. Publication-Quality and Reconstructible Figures*

262 SciTeX’s visualization module produced figures with consistent styling
263 and journal-ready dimensions across platforms. When saving figures, the
264 system generated a triplet consisting of a high-resolution PNG, a JSON
265 metadata file encoding axis bounds, geometry, and color specifications, and
266 a CSV file containing underlying plot data. Figures reconstructed using
267 `stx.plt.load()` reproduced the original output faithfully. These capabili-
268 ties confirm that SciTeX captures sufficient metadata for downstream verifi-
269 cation and exact regeneration.

270 *3.3. Universal Data Handling and Format-Agnostic I/O*

271 The `stx.io` module successfully loaded and saved more than thirty file
272 types through extension-based detection. Hierarchical formats such as HDF5
273 and Zarr were navigated interactively, and caching reduced repeated ac-
274 cess overhead. Cross-platform tests confirmed consistent behavior for CSV,
275 JSON, NumPy arrays, PyTorch tensors, and image formats. These results il-
276 lustrate how unified I/O simplifies data flow through analysis and manuscript
277 preparation.

278 *3.4. Integrated Literature Management*

279 The Scholar module retrieved metadata from CrossRef, Semantic Scholar,
280 and OpenAlex, stored PDFs using DOI-based indexing, and maintained syn-
281 chronized BibTeX entries for manuscripts authored in Writer. Filtering by

282 year, journal, and citation count behaved as expected. Integration tests con-
283 firmed that citations added in Writer consistently matched entries managed
284 by Scholar, eliminating drift between literature databases and LaTeX sources.

285 *3.5. Containerized Manuscript Compilation*

286 Manuscripts compiled identically across Linux, macOS, Windows Sub-
287 system for Linux, and HPC environments when built through SciTeX’s con-
288 tainerized LaTeX engines. Tectonic produced fast incremental builds while
289 latexmk and multi-pass compilation ensured compatibility with a wide range
290 of documents. Identical inputs produced byte-identical PDF outputs, vali-
291 dating that containerization eliminates environment-dependent inconsisten-
292 cies.

293 *3.6. Cross-Module Traceability*

294 SciTeX’s symlink-based architecture correctly synchronized figures, bibli-
295 ography files, and computational outputs across modules. Global identifiers
296 enabled reverse lookup queries such as linking figures to their generating
297 scripts or tracing bibliography entries to manuscript citations. Metadata
298 exported by Viz and IO was successfully used by Code and Writer, demon-
299 strating that provenance information flows consistently across the ecosystem.

300 *3.7. Self-Descriptive Validation*

301 This manuscript itself validates SciTeX’s integrated design: it was au-
302 thored in Writer, managed with Scholar, compiled in a containerized environ-
303 ment, and incorporates figures generated in Viz and data processed through
304 Code. The end-to-end workflow confirms that SciTeX can be used to docu-
305 ment the system within the system itself, providing a live demonstration of
306 its reproducibility guarantees.

307 **4. Discussion**

308 SciTeX addresses structural challenges in modern scientific research by
309 unifying manuscript preparation, figure generation, literature management,

310 and reproducible computation within a single, provenance-preserving ecosys-
311 tem. Contemporary workflows are shaped by practical frustrations: frag-
312 mented toolchains, inconsistent figure provenance, manual versioning, and
313 the cognitive overhead of switching between environments [1, 3]. SciTeX re-
314 frames these challenges not as isolated usability problems but as symptoms
315 of a deeper architectural gap—namely, the absence of a shared substrate
316 linking computational, visual, and written components of scientific work.

317 *4.1. Design Philosophy and Modularity*

318 A central principle of SciTeX is incremental adoption. Each module pro-
319 vides standalone value while integrating seamlessly into a shared project
320 structure. Researchers who begin with the Writer module obtain container-
321 ized reproducibility without modifying their computational workflow. Adding
322 Scholar ensures consistent bibliographic management. Viz enables provenance-
323 tracked, reconstructible figures. Code completes the pipeline by linking anal-
324 yses directly to manuscript outputs. This design lowers the activation energy
325 required to engage in reproducible practices [2, 20]. It democratizes access
326 to advanced computational workflows by providing high-level abstractions
327 that remain accessible to users unfamiliar with Git, containers, or Linux
328 environments, while still offering fine-grained control for expert users.

329 By embedding provenance directly into the system’s data structures,
330 SciTeX avoids the pitfalls of ad hoc reproducibility strategies that rely on
331 user discipline. Global identifiers and shared directories ensure that figures,
332 scripts, and bibliographies remain synchronized without users needing to
333 track these relationships manually. This approach transforms reproducibility
334 from an additional burden into a natural consequence of ordinary workflow.

335 *4.2. Positioning Relative to Existing Tools*

336 SciTeX occupies a unique position among research platforms by inte-
337 grating capabilities that existing solutions offer only in isolation. Overleaf
338 provides consistent LaTeX compilation but does not integrate analysis or
339 figure provenance. Zotero and Mendeley excel at literature management yet

340 remain disconnected from manuscript workflows. Jupyter notebooks enable
341 interactive computation but do not support structured writing or publication-
342 quality visualization. Manubot demonstrates the advantages of Git-based
343 collaborative writing, while visualization tools such as SigmaPlot offer in-
344 tuitive figure creation without provenance guarantees. R-based ecosystems
345 such as the Rockerverse provide end-to-end reproducibility within the R lan-
346 guage but do not extend across multi-language workflows [21].

347 SciTeX differs by providing a coherent architecture spanning writing,
348 computation, visualization, and literature management. Its modular design
349 supports independent usage, yet deeper integration emerges organically when
350 modules are combined. This positions SciTeX as a research operating envi-
351 ronment rather than a single-purpose tool, while still maintaining compati-
352 bility with existing toolchains rather than attempting to replace them.

353 4.3. *AI-Native Design*

354 As AI systems increasingly support scientific workflows, a platform’s abil-
355 ity to interoperate with automated agents becomes essential [24, 25]. SciTeX
356 incorporates AI-native design principles by exposing structured APIs, JSON-
357 based figure specifications, and typed function signatures that enable auto-
358 mated analysis, figure refinement, and manuscript revision. Workflow hooks
359 allow AI agents to monitor execution, interpret metadata, and propose mod-
360 ifications while ensuring that all AI-assisted operations remain traceable.
361 This design supports future research co-pilot systems capable of generating
362 figures, summarizing literature, conducting analyses, and drafting sections of
363 manuscripts while preserving researcher oversight and scientific integrity.

364 4.4. *Implications for Reproducibility*

365 The reproducibility crisis in science arises partly because researchers lack
366 integrated infrastructure for capturing computation, visualization, and writ-
367 ing as a coherent whole [28, 14]. SciTeX mitigates this by ensuring that
368 manuscripts compile identically across systems, that figures can be regener-
369 ated from metadata, and that analyses are executed within controlled en-

370 vironments. Its architecture aligns with FAIR principles and contemporary
371 reproducibility frameworks [11, 10]. By shifting reproducibility from a set
372 of optional best practices to an embedded property of the research work-
373 flow, SciTeX reduces the friction that often prevents rigorous methodological
374 documentation.

375 *4.5. Limitations*

376 Despite its integrated design, SciTeX has several limitations. Support for
377 HPC systems and SLURM-based workflows remains under active develop-
378 ment, with current functionality focused primarily on local and containerized
379 execution. Empirical evaluation of SciTeX’s impact on researcher productiv-
380 ity is ongoing, as the user base is still expanding. While the Writer module
381 simplifies LaTeX-based workflows, researchers unfamiliar with LaTeX may
382 still face an initial learning curve. The platform targets desktop research
383 environments, and mobile support has not been prioritized. Advanced sta-
384 tistical visualization beyond standard matplotlib capabilities may require
385 manual customization.

386 *4.6. Future Directions*

387 Future development will extend SciTeX’s AI-native capabilities, including
388 automated figure refinement, semantic consistency checking, and context-
389 aware citation recommendation. Deeper integration of HPC-native work-
390 flows, unified inspector interfaces across modules, and interactive provenance
391 graph visualization are planned. The long-term vision is a research co-pilot
392 capable of executing analyses, generating publication-ready figures, inter-
393 preting literature, and producing draft manuscripts while maintaining full
394 transparency and human oversight.

395 **5. Conclusions**

396 SciTeX represents both an architectural framework and a practical tool for
397 reproducible scientific research. By combining modular design with provenance-
398 aware infrastructure and AI-native workflows, it provides an environment

399 where researchers can focus on scientific reasoning rather than technical or-
400 chestration. This manuscript, prepared entirely within SciTeX, illustrates
401 the platform’s ability to describe, manage, and reproduce its own workflow.

402 References

- 403 [1] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Neder-
404 bragt, and Tracy K. Teal. Good enough practices in scientific com-
405 puting. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi:
406 10.1371/journal.pcbi.1005510.
- 407 [2] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig.
408 Ten simple rules for reproducible computational research. *PLoS Com-
409 putational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.
410 1003285.
- 411 [3] Christian T. Jacobs. Improving the reproducibility of L^AT_EX documents
412 by enriching figures with embedded scripts and data. *International Jour-
413 nal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.
- 414 [4] Haim Bar and HaiYing Wang. Reproducible science with L^AT_EX. *Journal
415 of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.
- 416 [5] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim
417 Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing
418 Dockerfiles for reproducible data science. *PLOS Computational Biology*,
419 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.
- 420 [6] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman,
421 Brian Granger, et al. Binder 2.0—reproducible, interactive, sharable
422 environments for science at scale. *Proceedings of the Python in Science
423 Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4af1f417-011.
- 424 [7] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slochower, Dongbo
425 Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open

- 426 collaborative writing with Manubot. *PLOS Computational Biology*, 15
427 (11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.
- 428 [8] Albert Krewinkel and Robert Winkler. Formatting open science: Ag-
429 ilely creating multiple document formats for academic manuscripts with
430 Pandoc Scholar. *PeerJ Preprints*, 5:e2648, 2017. doi: 10.7287/peerj.
431 preprints.2648v2.
- 432 [9] Markus Konkol, Daniel Nüst, and Laura Goulier. Publishing computa-
433 tional research—a review of infrastructures for reproducible and trans-
434 parent scholarly communication. *Research Integrity and Peer Review*, 5,
435 2020. doi: 10.1186/s41073-020-00095-y.
- 436 [10] Jonathan M. Mang, Hans-Ulrich Prokosch, and Lorenz A. Kapsner.
437 Reproducibility in 2023—an end-to-end template for analysis and
438 manuscript writing. *Studies in Health Technology and Informatics*, 302,
439 2023. doi: 10.3233/shti230064.
- 440 [11] The Turing Way Community. *The Turing Way: A Handbook for Re-*
441 *producible, Ethical and Collaborative Research*. Zenodo, 2022. doi:
442 10.5281/zenodo.3233853.
- 443 [12] Armin Ariamajd, Raquel López-Ríos de Castro, and Andrea Volka-
444 mer. PyPackIT: Automated research software engineering for scientific
445 Python applications on GitHub. *arXiv.org*, abs/2503.04921, 2025. doi:
446 10.48550/arxiv.2503.04921.
- 447 [13] Christian Schulz. Reproducible data citations for computational re-
448 search. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.
449 48550/arxiv.1808.07541.
- 450 [14] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Re-
451 producible research in R: A tutorial on how to do the same thing more
452 than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.

- 453 [15] Nikolas I. Krieger, Adam Perzynski, and Jarrod Dalton. Facilitating
454 reproducible project management and manuscript development in team
455 science: The projects R package. *PLoS ONE*, 14, 2019. doi: 10.1371/
456 journal.pone.0212390.
- 457 [16] Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data an-
458 alytical work reproducibly using R (and friends). *The American Statis-*
459 *tician*, 72(1):80–88, 2018. doi: 10.1080/00031305.2017.1375986.
- 460 [17] Juan Pedro Araque Espinosa et al. A continuous integration and web
461 framework in support of the ATLAS publication process. *Journal of*
462 *Instrumentation*, 16, 2020. doi: 10.1088/1748-0221/16/05/T05006.
- 463 [18] Michael Canesche, Roland Leißa, and Fernando Magno Quintão Pereira.
464 Preparing reproducible scientific artifacts using Docker. *arXiv.org*,
465 abs/2308.14122, 2023. doi: 10.48550/arxiv.2308.14122.
- 466 [19] Peter Kastrup. Tectonic: A modernized, self-contained L^AT_EX engine,
467 2024. URL <https://tectonic-typesetting.github.io/>.
- 468 [20] Kristina Wiebels and David Moreau. Leveraging containers for repro-
469 ducible psychological research. *Advances in Methods and Practices in*
470 *Psychological Science*, 4, 2021. doi: 10.1177/25152459211017853.
- 471 [21] Daniel Nüst, Dirk Eddelbuettel, Dom Bennett, Robrecht Cannoodt,
472 Dave Clark, et al. The Rockerverse: Packages and applications for con-
473 tainerisation with R. *The R Journal*, 12:437, 2020. doi: 10.32614/
474 RJ-2020-007.
- 475 [22] Karthik Ram. Git can facilitate greater reproducibility and increased
476 transparency in science. *Source Code for Biology and Medicine*, 8:7,
477 2013. doi: 10.1186/1751-0473-8-7.
- 478 [23] Karl-Ludwig Besser. Review response template. [https://www.](https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd)
479 [overleaf.com/latex/templates/review-response-template/](https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd)
480 [tmbvmjstxwrd](https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd), 2025.

- [24] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.
- [25] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.
- [26] OpenLens Team. OpenLens AI: Fully autonomous research agent for health informatics. *arXiv preprint arXiv:2509.14778*, 2025.
- [27] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.
- [28] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

6. Resource Availability

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- The SciTeX Python package is available via PyPI: `pip install scitex`

- 505 • Documentation is available at: <https://docs.scitex.ai>
- 506 • The cloud platform is accessible at: <https://scitex.ai>
- 507 • This manuscript's source files are included in the SciTeX Writer repos-
508 itory as a demonstration
- 509 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

510 All code is released under the GNU General Public License v3.0 (GPL-
511 3.0), ensuring open access and modification rights. The platform follows
512 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
513 and code artifacts.

514 Any additional information required to reanalyze the data reported in
515 this paper is available from the lead contact upon request.

516 **Ethics Declarations**

517 All study participants provided their written informed consent ...

518 **Author Contributions**

519 Y.W., T.Y., and D.G. conceptualized the study ...

520 **Acknowledgments**

521 This research was funded by **funding bodies here**

522 **Declaration of Interests**

523 The authors declare that they have no competing interests.

524 **Declaration of Generative AI in Scientific Writing**

525 The authors employed large language models such as Claude (Anthropic
526 Inc.) for code development and complementing manuscript’s English lan-
527 guage quality. After incorporating suggested improvements, the authors
528 meticulously revised the content. Ultimate responsibility for the final content
529 of this publication rests entirely with the authors.

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

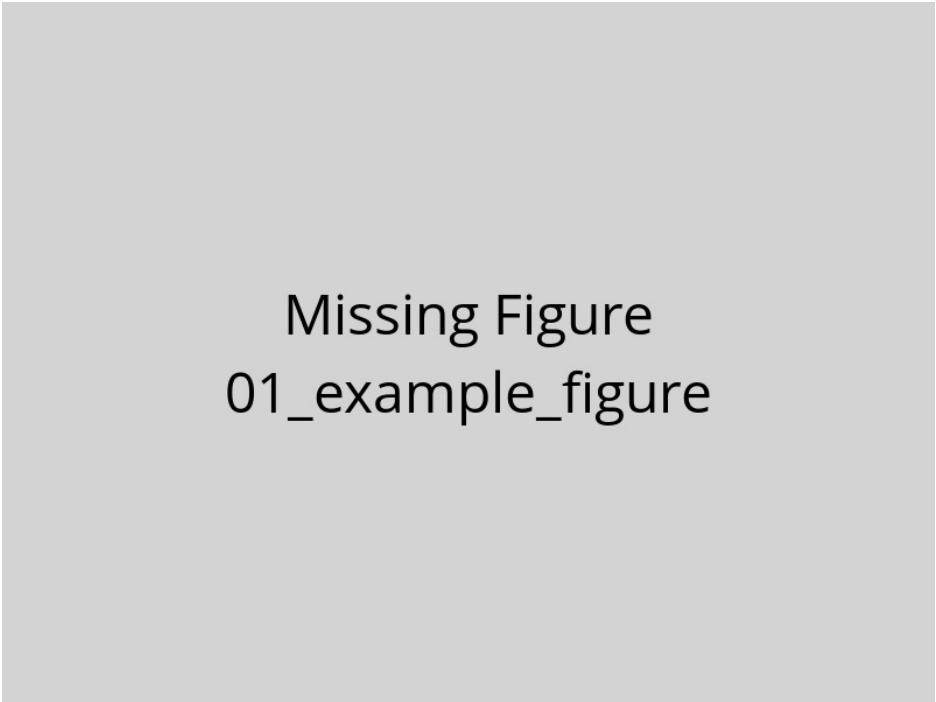
Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

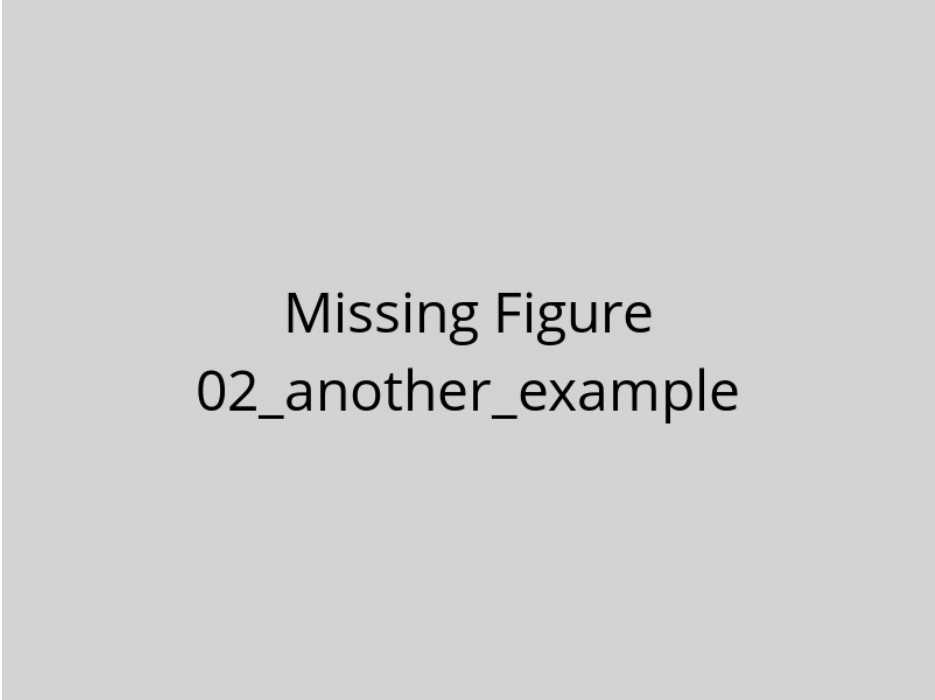
Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.



Missing Figure
01_example_figure

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

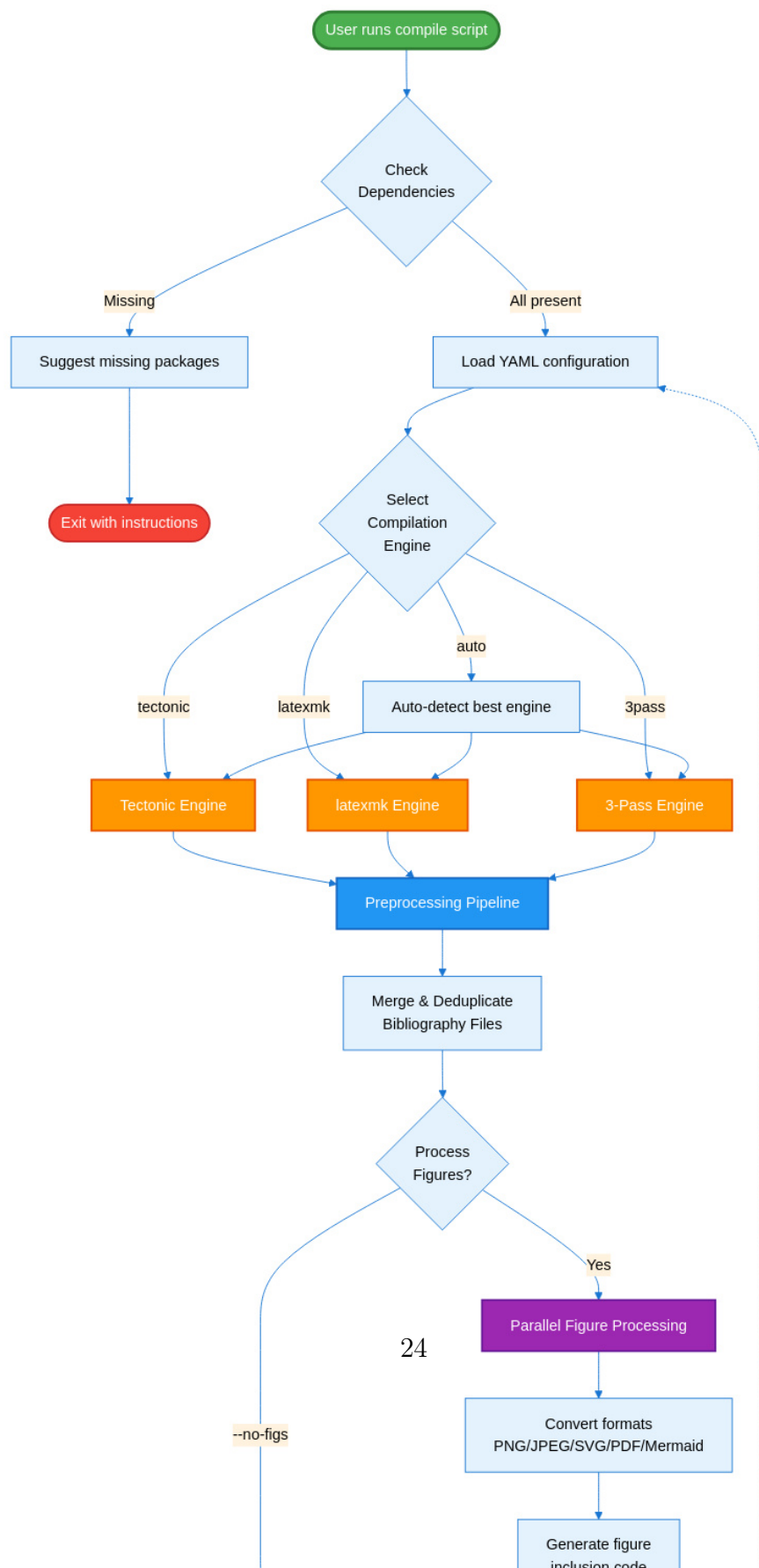




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

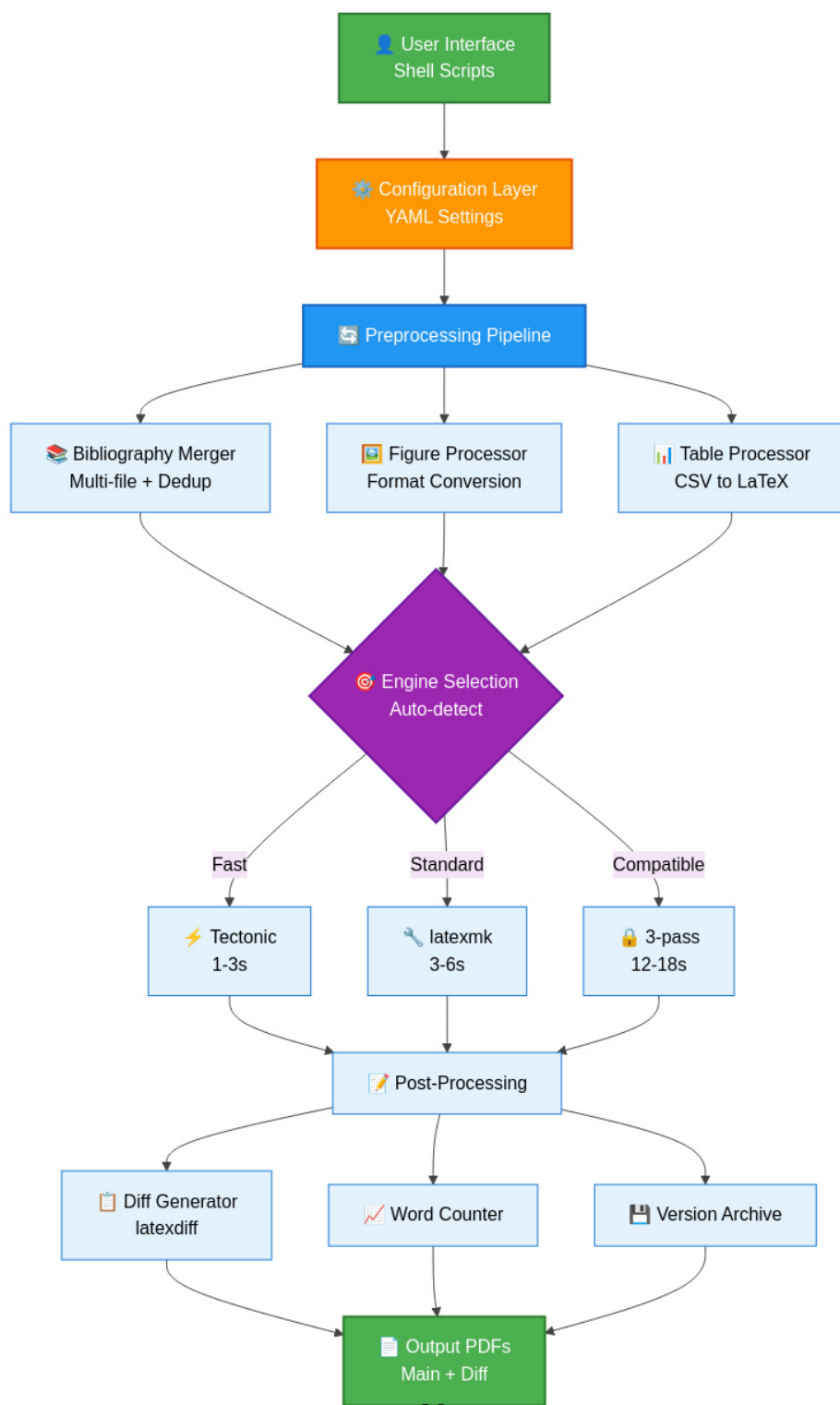


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

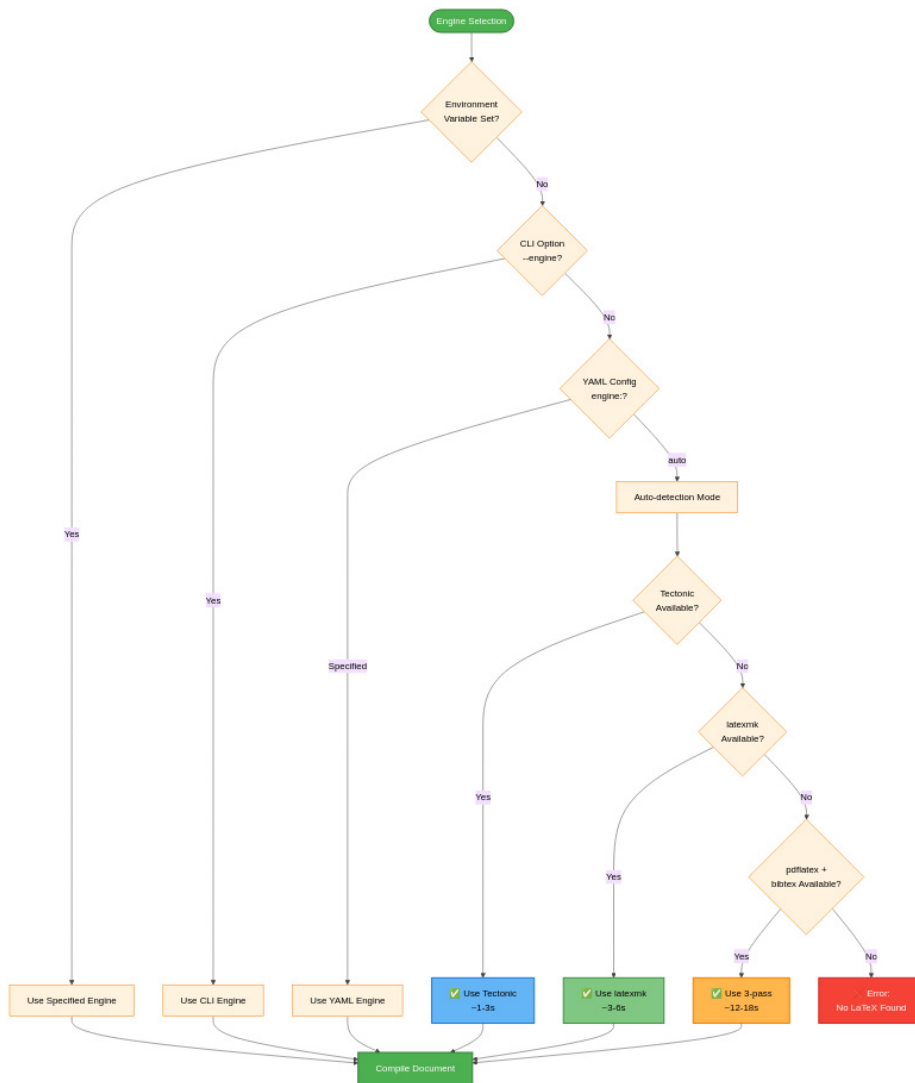


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to `ltxmk` for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

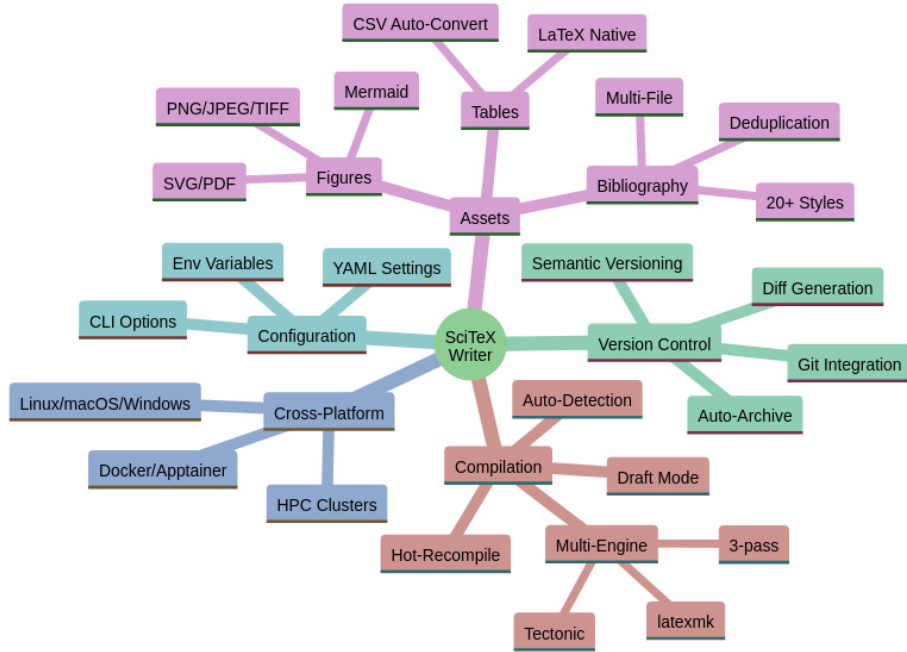


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).